A network that models $d\sigma/dm_{hh}$

Plan: bin integrals by Gauss–Legendre quadrature, smooth activations, and a curvature penalty

P. Bhat (Fermilab), N. De Filippis (Politecnico di Bari),
K.F. Johnson, H.B. Prosper, R. Starnes (Florida State University)

November 2026 (addendum: 4 November)

Abstract

`HEFTNet` is trained on the cross section per 15 GeV bin, at the bin centres, so it models the *bin contents*, not the density $d\sigma/dm_{hh}$. Between bin centres nothing constrains it. We propose to make the network a density. The network's 23 coefficient functions become densities $a_i(m)$, and the loss compares their *integral* over each bin with POWHEG, exactly as for the smooth spline model. The integrals are computed on the GPU at every step by Gauss–Legendre quadrature, with nodes and weights generated in PyTorch. Because a_i depends only on m, this costs about 640 evaluations of a one-input network per step, *whatever the batch size*. The plan also covers three related questions. Which activation gives a smooth density: ReLU, sine, tanh or SiLU? Should the threshold factor be built in? And how do we suppress structure inside a bin, which bin integrals cannot see? Decisions for you are collected in section 11.

The proposal in one paragraph

Keep $\dfrac{d\sigma}{dm}(m;\theta) = \sum_{i=1}^{23} C_i(\theta)\, a_i(m)$ with $a_i(m) = \phi(m)\, P_i(x)\, e^{Q_i(x)}$, $x = (m - 255\,\text{GeV})/1500\,\text{GeV}$. Predict bin k by $f_k(\theta) = \sum_i C_i(\theta) A_{ik}$, with $A_{ik} = \int_{\text{bin }k} a_i\, dm \approx \sum_g w_{kg}\, a_i(m_{kg})$, using $n = 8$ Gauss–Legendre nodes m_{kg} and weights w_{kg} per bin. The training rows, targets, losses, `drwho` tools and the fine-tune are unchanged: only `forward` changes. Options: a smooth activation (sine, tanh, SiLU), the threshold factor ϕ, and a small penalty on the curvature of the a_i. Evaluation: the merged-bin test, the spread of an ensemble between bin centres, and direct comparison with the spline model's $d\sigma/dm$.

Contents

1 The problem **1**
1.1 What `HEFTNet` models now . . . 1
1.2 Why randomising m within the bin does not help . . . 2

2 The idea: a density, compared through its bin integrals **2**

3 Gauss–Legendre quadrature, from the beginning **4**
3.1 The idea . . . 4
3.2 Computing nodes and weights in PyTorch: the Golub–Welsch method . . . 4
3.3 How many nodes? . . . 5

4 The threshold factor **6**

5 The activation: smoothness of the density **6**
5.1 ReLU gives kinks 6
5.2 Smooth alternatives 6

6 What bin integrals cannot see, and a curvature penalty **7**
6.1 The null space 7
6.2 A penalty on curvature 7

7 Training **7**

8 Code **8**

9 Tests and evaluation **9**

10 Order of work **10**

11 Decisions for you **10**

12 Addendum (4 November): decisions, and what the toy test changed **10**

1 The problem

1.1 What `HEFTNet` models now

Each training row is $(x_k, \theta_s;\, t_{sk})$: the bin centre, the HEFT point and the POWHEG cross section in the bin. The network is evaluated at the bin centre only, and its output is compared with t_{sk}. So the network learns the function "content of the bin whose centre is at x", and it learns it *only at the 80 centres*. Two consequences follow.

1. **At a centre, it is not** $15\ \mathbf{GeV} \times d\sigma/dm$**.** The bin content is the integral of the density over the bin, which differs from 15 GeV times the value at the centre by about $\frac{1}{24}(15\ \mathrm{GeV})^2\, f''$. Where the density is curved, this is not small. For the SM, using the smooth model's density, the difference is -12% in the first bin (255–270 GeV), -0.8% in the second and -0.3% at 300 GeV. For the most cancelling test spectrum it is -14%, $+3.4\%$ and $+1.8\%$. Compare the Poisson noise there, about 3%.

2. **Between centres, it is unconstrained.** Whatever the network does between centres costs nothing in the loss. Below 400 GeV the 20 networks of the ensemble disagree 20–90 times more between centres than at them (`docs/paper/fig_interpolation.pdf`).

1.2 Why randomising m within the bin does not help

A natural idea is to draw m uniformly in $[255, 1455)$, attach the content t_k of the bin it falls in (perhaps with added Poisson noise), and hope the network averages this into a smooth density. It will not, and the reason is worth seeing.

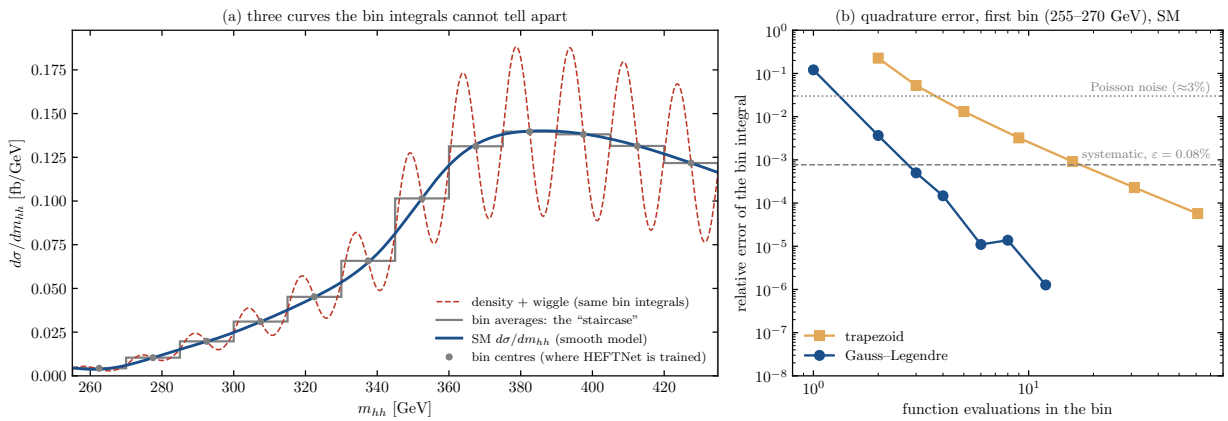


Figure 1: (a) The SM density of the smooth model (blue), its bin averages (grey: the staircase a network trained on bin values tends towards), and a density with a wiggle added in every bin (red, dashed). All three have *identical* bin integrals: the data cannot tell them apart. (b) Relative error of the integral over the first bin, the hardest one, against the number of function evaluations, for the trapezoid rule and Gauss–Legendre. The flattening of Gauss–Legendre near 10^{-5} is a property of the test integrand: it is itself a cubic spline with knots inside the bin. A smooth network has no such knots.

What a network trained on noisy targets converges to

With the squared loss, the best function (the minimiser of the risk, by the calculus of variations) is the *conditional mean* of the target given the inputs:

$$g^*(m,\theta) = \mathbb{E}[\,t \mid m,\theta\,]. \tag{1}$$

If m lies in bin k, the target is t_k, plus noise of zero mean. So for *every* m in bin k,

$$g^*(m,\theta) = t_k(\theta) \qquad \text{(a constant across the bin)}. \tag{2}$$

The optimum is the histogram itself, a **staircase** (grey in fig. 1a). With the L1 loss the optimum is the conditional median, which is the same staircase. Randomising m only shows the network more points on the flat top of each step. The added noise averages to zero, so it does not move the optimum either. It only acts as a mild regulariser whose strength has no physical meaning, and it counts the POWHEG Poisson noise twice.

Any smoothness the network shows would come from its *inability* to draw sharp steps: a staircase with rounded corners. That is the bin content smoothed once more over 15 GeV, which lowers peaks and fills dips, and how much it does so depends on width, depth, training time and seed rather than on physics.

2 The idea: a density, compared through its bin integrals

The data are bin integrals, so the model should be a density and the loss should compare its integrals with the data:

$$f_k(\theta) = \int_{e_k}^{e_{k+1}} \frac{d\sigma}{dm}\, dm = \sum_{i=1}^{23} C_i(\theta)\, A_{ik}, \qquad A_{ik} = \int_{e_k}^{e_{k+1}} a_i(m)\, dm. \tag{3}$$

This is what the smooth model does, with splines. Here the a_i come from the network. Statisticians call such a model, one where the observations are integrals (or sums) of an unobserved smooth function, a *composite link model* [1].

Why this is cheap

The 23 functions a_i depend on m only, not on θ. So at each step we evaluate the network at a *fixed* set of nodes, 8 per bin, 640 in all, and form the table A_{ik} [80, 23]. Each row (θ_s, k) of the batch then needs only the dot product of $\boldsymbol{C}(\theta_s)$ with row k of that table. Today the network is evaluated once per batch row (2,048 times per step). With the density model it is evaluated 640 times per step, whatever the batch size.

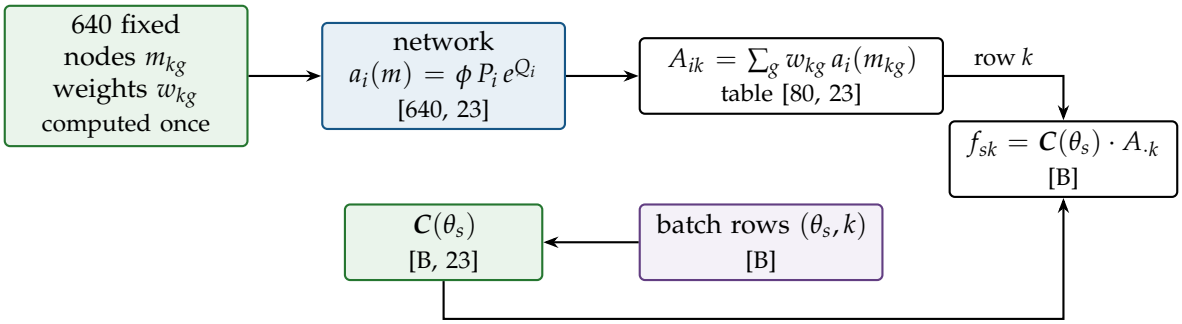


Figure 2: The forward pass of the density model. Green: fixed (nodes, weights, monomials). Blue: learned. The batch only selects rows of the table A_{ik}.

Units, and a warm start. We let the network output $\hat{a}_i(m) = 15\ \text{GeV} \times a_i(m)$, the density "per 15 GeV". Then

$$A_{ik} = \sum_g \frac{w_{kg}}{15\ \text{GeV}}\, \hat{a}_i(m_{kg}), \tag{4}$$

and $\hat{a}_i$ has the same size as today's network output. Any trained `HEFTNet`, e.g. the Huber-fine-tuned `runs/FT_huber_d2`, can then be loaded into the density model as a starting point (without ϕ; section 4).

3 Gauss–Legendre quadrature, from the beginning

3.1 The idea

A quadrature rule approximates an integral by a weighted sum of function values:

$$\int_{-1}^{1} f(x)\, dx \approx \sum_{g=1}^{n} w_g\, f(x_g). \tag{5}$$

The trapezoid rule fixes the nodes x_g (equally spaced) and chooses only the weights. That leaves n free numbers, enough to integrate polynomials of degree $n-1$ exactly; in practice the composite trapezoid rule is exact only for straight lines, and its error falls like h^2 with the spacing h. Gauss's idea (1814) was to choose the nodes *as well*. With $2n$ free numbers one can hope to integrate exactly all polynomials of degree $2n-1$, and this is possible.

Why n nodes integrate degree $2n-1$ exactly

Let P_n be the Legendre polynomial of degree n. The Legendre polynomials are *orthogonal* on $[-1,1]$: $\int_{-1}^{1} P_n\, q\, dx = 0$ for every polynomial q of degree less than n. Take the nodes x_g to be the n zeros of P_n, and the weights w_g to be the integrals of the Lagrange interpolating polynomials through those nodes, so that the rule is exact for every polynomial of degree $\le n-1$.
Now take any polynomial p of degree $\le 2n-1$ and divide it by P_n: $p = q\,P_n + r$, where the quotient q and the remainder r both have degree $\le n-1$. Then:

- exactly: $\int p\,dx = \int qP_n\,dx + \int r\,dx = 0 + \int r\,dx$, by orthogonality;
- by the rule: $\sum_g w_g\, p(x_g) = \sum_g w_g\, r(x_g)$, because $P_n(x_g) = 0$ at every node;
- and $\sum_g w_g\, r(x_g) = \int r\,dx$, because the rule is exact for degree $\le n-1$.

So the rule gives $\int p\,dx$ exactly. For a smooth f the error is

$$E_n = \frac{(b-a)^{2n+1}(n!)^4}{(2n+1)\,[(2n)!]^3}\, f^{(2n)}(\xi), \qquad \xi \in (a,b), \tag{6}$$

against $-\frac{(b-a)\,h^2}{12} f''(\xi)$ for the composite trapezoid rule with spacing h.

3.2 Computing nodes and weights in PyTorch: the Golub–Welsch method

PyTorch has `torch.trapezoid` but no Gauss–Legendre routine. We do not need one: the nodes and weights are computed *once*, before training, and then sit on the GPU as two constant tensors. They can nevertheless be computed in PyTorch, in a few lines, by the method of Golub and Welsch [2].

Nodes as eigenvalues

Orthonormal polynomials obey a three-term recurrence. For the normalised Legendre polynomials p_k it reads

$$x\,p_k(x) = \beta_{k+1}\,p_{k+1}(x) + \beta_k\,p_{k-1}(x), \qquad \beta_k = \frac{k}{\sqrt{4k^2-1}}. \tag{7}$$

Write it for $k = 0,\dots,n-1$ as a matrix equation for the vector $\boldsymbol{p} = (p_0,\dots,p_{n-1})$:

$$x\,\boldsymbol{p}(x) = J\,\boldsymbol{p}(x) + \beta_n\,p_n(x)\,\boldsymbol{e}_n, \tag{8}$$

where J is the symmetric tridiagonal matrix with zeros on the diagonal and $\beta_1,\dots,\beta_{n-1}$ beside it, and $\boldsymbol{e}_n$ is the last unit vector. At a zero x_g of p_n the last term vanishes, leaving $J\,\boldsymbol{p}(x_g) = x_g\,\boldsymbol{p}(x_g)$: **the nodes are the eigenvalues of** J. One can show that the weights are $w_g = 2\,v_{1g}^2$, where v_{1g} is the first component of the normalised eigenvector g, and 2 is $\int_{-1}^{1} dx$.

```
import torch

def gauss_legendre(n, dtype=torch.float64):
    '''n-point Gauss-Legendre nodes x and weights w on [-1, 1] (Golub-Welsch).'''
    k   = torch.arange(1, n, dtype=dtype)
    b   = k / torch.sqrt(4 * k**2 - 1)            # beta_1 ... beta_{n-1}
```

```
    J   = torch.diag(b, 1) + torch.diag(b, -1)     # symmetric tridiagonal matrix
    x, V = torch.linalg.eigh(J)                    # eigenvalues (sorted), eigenvectors
    w   = 2 * V[0]**2                              # first component of each eigenvector
    return x, w

def bin_nodes(edges, n=8):
    '''Nodes m [K*n] and weights w [K*n] for the bins [edges[k], edges[k+1]].'''
    x, w = gauss_legendre(n)
    e = torch.as_tensor(edges, dtype=torch.float64)
    c, h = 0.5 * (e[1:] + e[:-1]), 0.5 * (e[1:] - e[:-1])  # centres, half-widths
    m  = (c[:, None] + h[:, None] * x[None]).reshape(-1)   # map [-1,1] onto each bin
    wt = (h[:, None] * w[None]).reshape(-1)                # dm = h dx
    return m, wt
```

The computation is done in double precision, so the nodes and weights are exact to the last bit, and then converted to float32. We will check them against `numpy.polynomial.legendre.leggauss` (agreement to 10^{-15}) and against exact integrals of polynomials of degree $2n-1$.

3.3 How many nodes?

Figure 1b and table 1 show the relative error of the bin integral for realistic integrands, the smooth model's own densities. The first bin is the hardest, because the threshold factor $\phi = \sqrt{1-4m_h^2/m^2}$ bends sharply there: $|\phi''/\phi| \approx 0.01$ GeV^{-2} at 255 GeV.

		trapezoid, sub-intervals per bin			Gauss–Legendre, nodes per bin		
spectrum	bin (GeV)	1	4	15	2	4	8
SM	255–270	23%	1.3%	0.09%	0.4%	0.015%	0.0014%
SM	300–315	0.6%	0.04%	0.003%	0.01%	3×10^{-6}	2×10^{-7}
most cancelling (ID 917)	255–270	26%	1.4%	0.10%	0.6%	0.028%	0.0025%
most cancelling (ID 917)	300–315	2.6%	0.10%	0.007%	0.24%	0.006%	0.0003%

Table 1: Relative error of the integral over one bin, for the smooth model's density. For comparison: Poisson noise near threshold $\approx 3\%$; the systematic $\varepsilon \approx 0.08\%$.

Why the quadrature must be better than the noise

The loss never sees the exact integrals, only the quadrature sums. The network will learn whatever density makes the *sums* match POWHEG. A quadrature error therefore does not show up in the loss or in the bin predictions; it shows up as a bias in the learned density. With 8 Gauss–Legendre nodes per bin the error is below 3×10^{-5} in every case tested, well below the systematic floor. The trapezoid rule would need about 1 GeV spacing near threshold just to reach the floor ($\approx 10^{-3}$, column "15"). An earlier estimate of mine in the chat (0.03% at 1 GeV) was too optimistic by a factor of 3.

Proposal: 8 Gauss–Legendre nodes in every bin (640 nodes), and a check that 16 nodes change nothing.

4 The threshold factor

At $m = 2m_h = 250$ GeV the two-body phase space closes, and $d\sigma/dm$ vanishes like the Higgs velocity in the hh rest frame, $\phi(m) = \sqrt{1 - 4m_h^2/m^2}$. Today's network has no reason to know this. Multiplying by ϕ,

$$\hat{a}_i(m) = \phi(m)\, P_i(x)\, e^{Q_i(x)}, \tag{9}$$

leaves the network only the smooth remainder to describe. For the spline model this changed nothing inside 255–1455 GeV, but made the extrapolation to 240–255 GeV 8% high instead of 55% high. With ϕ on, a warm start from an existing network is poor near threshold (there $\phi \approx 0.2$–0.5), so training should start from scratch. With ϕ off, the warm start is a quick first test.

5 The activation: smoothness of the density

5.1 ReLU gives kinks

P is a network of one input. With ReLU, each hidden unit is either off or linear in its input, so $P_i(x)$ is *piecewise linear* in x, with a kink wherever a unit switches. The final tanh and the envelope e^Q round the kinks somewhat, but the derivative of $d\sigma/dm$ still jumps at places chosen by the training, not by the physics. For bin integrals this hardly matters. For a density meant to be read between bin centres, and differentiated, it does.

5.2 Smooth alternatives

activation	smoothness	behaviour
ReLU (today)	kinks	piecewise linear; baseline
tanh	smooth	saturates; classic choice for smooth regression
SiLU, $x/(1+e^{-x})$	smooth	ReLU-like for large x, smooth near 0; trains like ReLU
sine	smooth, oscillatory	see below

Sine. `heftnet.py` already defines a `Sin` module, and `ACTIVATION = 'Sin'` works with the architecture as it stands. Sine networks ("SIRENs" [3]) are smooth to all orders, and are popular because, with a frequency factor $\sin(30\,Wx)$ in the first layer and a special initialisation, they represent fine detail very well. Fine detail is exactly what we do *not* want: a sine network can oscillate inside a bin at no cost in the loss (section 6). With PyTorch's default initialisation and no frequency factor the first layer's weights lie in $(-1, 1)$, and our input x lies in $[0, 0.8]$, so the argument of each first-layer sine changes by less than one radian across the whole mass range. The network then starts in the low-frequency regime and behaves like a smooth basis, much like tanh. That is the regime we propose. Training can raise the frequencies, which the curvature penalty of the next section guards against. Outside the training range a sine network keeps oscillating, another reason not to use the network beyond 1455 GeV.

Proposal: a switch `ACTIVATION` $\in$ {`'nn.ReLU'`, `'Sin'`, `'nn.Tanh'`, `'nn.SiLU'`}, all four trained and compared (section 9).

6 What bin integrals cannot see, and a curvature penalty

6.1 The null space

Take any function $u(m)$ whose integral over every bin is zero, for example $u(m) = c_k \sin(2\pi(m - e_k)/15\,\text{GeV})$ in bin k. Adding u to any a_i changes no prediction, no loss and no gradient. The red curve of fig. 1a is such a case. In linear-algebra language, u is in the *null space* of the map from densities to bin integrals. The data say nothing about that part of the density. Something else must: the smoothness of the model.

For the spline model the something else is explicit: the splines cannot oscillate faster than the knot spacing, and the smoothness prior penalises curvature with a strength set by the evidence. For the network it is implicit, in the architecture and the training, and therefore uncontrolled. This is the deeper reason for the large spread of the ensemble between bin centres.

6.2 A penalty on curvature

We propose an explicit, optional penalty: on a uniform grid m_j with spacing $h = 1$ GeV from 255 to 1455 GeV (1,201 points), the second differences $\Delta^2 a_{ij} = a_i(m_{j+1}) - 2a_i(m_j) + a_i(m_{j-1}) \approx h^2 a_i''(m_j)$, and

$$R_{\text{curv}} = \lambda \, \frac{1}{23} \sum_{i=1}^{23} \frac{\sum_j \left(L^2\,\Delta^2 a_{ij}/h^2\right)^2}{\sum_j a_{ij}^2}, \qquad L = 100\ \text{GeV}. \tag{10}$$

Each term is the mean squared curvature of a_i on the scale L, relative to the size of a_i, so that functions of very different sizes are treated alike and λ is dimensionless. The denominator is held fixed in each step (`detach`), so the penalty only discourages curvature and does not shrink the functions. The wiggle of fig. 1a has $L^2|a''|/|a|$ of order $(2\pi L/15)^2 \approx 2 \times 10^3$; a physical a_i, varying on scales of 20–100 GeV, of order 1–25. So a small λ kills null-space wiggles while leaving the physics alone. This is the network version of the spline model's P-spline prior [4]. It costs 1,201 more evaluations of the one-input network per step.

Proposal: `CURV_LAMBDA` $\in \{0, 10^{-4}, 10^{-3}, 10^{-2}\}$; adopt the largest value that leaves the test-spectrum $-2\ln\lambda$ unchanged (within 0.005).

7 Training

Nothing about the data changes: the rows, the targets (pb per bin) and the splits are those of `heft_train.ipynb`. Because the density model takes the same input rows $[N, 6]$ and returns the same kind of output (pb per bin), the `drwho` datasets, loaders, monitor and `compute_avg_loss`, and `finetune.finetune`, all work unchanged.

1. **Stage 1**: Adam with the L1 loss, as now, from scratch (or warm start with ϕ off).
2. **Stage 2**: the fine-tune (`heft_finetune.ipynb`) with the Huber loss of the pulls, with `variance_from='model'`, i.e. the Poisson variance of the model's prediction. Weighting by the observed counts biases fits low (`docs/smooth/smooth_results.pdf`), and this was never done for the networks.
3. The curvature penalty, if used, is added to the loss in both stages.

The bin index k of a row is recovered from its input x as $k = \text{round}(100x - 0.5)$, exactly as in `heft_smooth.ipynb`.

8 Code

- **New module** `dihiggs/quadrature.py`: `gauss_legendre(n)` and `bin_nodes(edges, n)` as above, with tests.
- **New class** `HEFTDensity` **in** `dihiggs/heftnet.py`, a subclass of `HEFTNet`, so that every saved `HEFTNet` still loads as before:

```
class HEFTDensity(HEFTNet):
    '''HEFTNet as a density: forward(x) returns the INTEGRAL over the bin of x.'''
    def __init__(self, n_nodes=8, threshold=True,
                 edges=255.0 + 15.0 * np.arange(81), **kw):
        super().__init__(**kw)
        m, w = bin_nodes(edges, n_nodes)
        # register_buffer: tensors that move with model.to(device) and are saved
        # with the model, but are not trained
        self.register_buffer('m_nodes', m.float())
        self.register_buffer('w_nodes', (w / 15.0).float())   # output is "per 15 GeV"
        self.n_bins, self.n_nodes, self.threshold = len(edges) - 1, n_nodes, threshold

    def density15(self, m):
        '''15 GeV x a_i(m) at the masses m (GeV): [len(m), 23].'''
        a = self.coeffs((m - 255.0) / 1500.0)          # P * exp(Q), as before
        if self.threshold:
            a = a * torch.sqrt(1 - 4 * 125.0**2 / m**2)[:, None]
        return a

    def forward(self, x):
        # 1. the table of bin integrals A [n_bins, 23], from the fixed nodes
        a = self.density15(self.m_nodes) * self.w_nodes[:, None]
        A = a.view(self.n_bins, self.n_nodes, 23).sum(dim=1)
        # 2. each row picks its bin k and its 23 monomials C_i(theta)
        k = torch.round(x[:, 0] * 100 - 0.5).long()
        C = self.monomials(x[:, 1:])                       # the 23 C_i(theta), [B, 23]
        return (C * A[k]).sum(dim=1)
```

- The monomials, now written inside `HEFTNet.forward`, move to a method `monomials(theta)` that both classes use (no change in behaviour; checked against `smooth.heft_monomials`).
- A method `dsigma_dm(theta, m)` for the density at any m, used by `heft_test.ipynb`, the figures, and `smooth_common.network_density`.
- **Notebooks**: switches in `heft_train.ipynb` and `heft_finetune.ipynb` (table 2); run folders `runs/DENS_<activation>[_thr][_curv<lambda>]`.

9 Tests and evaluation

Before any training: a toy with a known answer. `code/smooth_toy.py` generates spectra of 40,000 Poisson events from three known smooth coefficient functions. We train the density network on its bins and compare the learned $a_i(m)$ with the truth *between* bin centres, together with a bin-value network as the contrast. This tests the whole idea where the answer is known.

switch	values	meaning
`MODEL`	`'bins'`, `'density'`	today's `HEFTNet`, or `HEFTDensity`
`N_NODES`	8 (16 as a check)	Gauss–Legendre nodes per bin
`THRESHOLD`	`True`, `False`	multiply by $\phi(m)$
`ACTIVATION`	`'nn.ReLU'`, `'Sin'`, `'nn.Tanh'`, `'nn.SiLU'`	hidden-layer activation
`CURV_LAMBDA`	$0, 10^{-4}, 10^{-3}, 10^{-2}$	curvature penalty eq. (10)
`WARM_START`	`None` or a parameter file	initialise from a trained `HEFTNet`

Table 2: Notebook switches.

On the POWHEG spectra.

1. **Accuracy**: mean $-2\ln\lambda$ per bin on the 1,000 test spectra, below and above 555 GeV, against the smooth model (1.566, 1.100) and the bin-value network (1.561, 1.121).
2. **Calibration**: pulls with the model-error term, overall and by cancellation fifth.
3. **Inside a bin, the merged-bin test**: train on the training spectra merged into 30 GeV bins (merging pairs of rows of a spectrum adds their counts, so the data stay Poisson; the density model just uses 30 GeV edges), predict the 15 GeV bins of the test spectra. The spline model passes above 360 GeV ($\chi^2 \approx 1$); a bin-value model cannot.
4. **Between bin centres**: an ensemble of 10 seeds for the best configuration; the relative spread of $d\sigma/dm$ between centres, against that at the centres (`fig_interpolation` updated).
5. **Against the spline model**: $d\sigma/dm$ and the $a_i(m)$ of the two models on a 0.5 GeV grid.
6. **Checks**: 8 against 16 nodes; the extrapolation to 240–255 GeV with ϕ.

Success criteria

Accuracy no worse than today's network; the merged-bin test passed above 360 GeV, like the spline model; the ensemble spread between bin centres no larger than at the centres; and a density that agrees with the spline model's within the two models' uncertainties.

10 Order of work

1. `quadrature.py` and its tests; `monomials()` refactor; `HEFTDensity`; a test that `forward` equals the bin integrals of `dsigma_dm` computed independently (CPU, here).
2. The toy (CPU or the 5090).
3. Notebook switches; one short run per activation with ϕ on and `CURV_LAMBDA` = 0 to check that training is stable (5090).
4. The grid: 4 activations $\times$ $\{\phi$ on$\}$ $\times$ 4 values of λ, stage 1 + stage 2 (5090; each run as long as today's).
5. Merged-bin test and ensemble for the best configuration; figures; a short results note.

11 Decisions for you

Decisions before I proceed

1. **Gauss–Legendre with 8 nodes per bin**, computed in PyTorch by Golub–Welsch, rather than the trapezoid rule. Agreed?
2. **Network output "per 15 GeV"** ($\hat{a}_i = 15\ \text{GeV} \times a_i$), so that existing networks can serve as a warm start. Agreed?
3. **Threshold factor ϕ on** by default, training from scratch. Agreed?
4. **Activations**: compare ReLU, sine (low frequency, default initialisation), tanh and SiLU. Any to add or drop?
5. **Curvature penalty** eq. (10) with a scan of λ. Agreed, or would you rather start without it?
6. **Code layout**: new `dihiggs/quadrature.py`; `HEFTDensity` as a subclass in `heftnet.py`; switches in the existing `heft_train.ipynb` and `heft_finetune.ipynb` rather than new notebooks. Acceptable?
7. **The toy test first**, before any POWHEG run. Agreed?

12 Addendum (4 November): decisions, and what the toy test changed

Decisions. (1) Gauss–Legendre, 8 nodes per bin. (2) The network models the density $a_i(m)$ itself, in pb/GeV; no warm start from the trained networks, so the "per 15 GeV" convention of section 2 is dropped. (3) Threshold factor on. (4) ReLU dropped; SiLU is the default, tanh and sine are options. (5) No curvature penalty for now. (6) Code layout as proposed. (7) The toy test first.

Two changes forced by the toy (`code/density_toy.py`).

1. **The network's input variable.** With HEFTNet's $x = (m - 255)/1500$ the density network could not fit the threshold region: χ^2 per bin of 6–13 in the first four bins, 1.7 overall. All the structure of 255–435 GeV lies in the first 12% of the range of x, and a smooth network cannot bend that sharply there. With

$$z = 2\sqrt{\frac{m - 250\ \text{GeV}}{1205\ \text{GeV}}} - 1, \tag{11}$$

   which, like the spline knots, gives 255–435 GeV a third of the input range, χ^2 per bin is ≈ 1 in every bin. This is now the default (`input_map='sqrt'`).

2. **The optimiser.** Full-batch Adam converges slowly on this problem (χ^2 per bin 1.74 after 15,000 steps). L-BFGS, a quasi-Newton method, converges in about a minute on a laptop, but only in double precision: in single precision its line search stalls. A full-batch evaluation of the density model is cheap (the network sees only the 640 nodes), so L-BFGS is practical here. New: `finetune.finetune_lbfgs` and the switch `OPTIMIZER='lbfgs'` in `heft_finetune.ipynb`.

Toy results. Three seeds per model; all reach χ^2 per bin ≈ 1. The "bins" model is the same network with the bin integral replaced by 15 GeV times the value at the bin centre (a 1-node rule), which is what HEFTNet assumes.

model	r.m.s. error of $a_i(m)$ / max$\lvert a_i\rvert$ (%) 0.5 GeV grid	bin centres	255–345 GeV	bias, 255–285 GeV (%) mean ± s.d. of seeds
density, SiLU	0.53	0.53	1.02	$+0.07 \pm 0.07$
density, tanh	0.51	0.51	1.09	
density, sine	0.36	0.36	0.93	$+0.38 \pm 0.25$
bins (1 node), SiLU	0.69	0.70	1.58	$+0.46 \pm 0.04$
spline model (reference)	0.16	0.16	0.44	+0.16

What the toy does and does not show

- The plan's criterion, a density network at least twice as accurate as the bin-value network, is **not** met in this toy: with a smooth activation, the square-root input and a good optimiser, the bin-value network is only modestly worse (0.69% against 0.53%). Neither shows a larger error between bin centres than at them. The toy is too easy to reproduce the spread of the old ensemble between centres; the factorial test below, on a realistic toy, does.
- What the integral removes is a **bias** where the density is curved: near threshold the bin-value network is wrong by +0.46%, the same in every seed (12 times their spread); the density network is unbiased there. In the toy the threshold curvature is mild. For POWHEG it is larger: for the SM, 15 GeV times the density at the centre of the first bin is 12% below the bin integral, so the bias of a bin-value model will matter there.
- The networks are 2–3.5 times less accurate than the spline model, which has a smoothness prior; their errors are smooth waves on scales of about 100 GeV, in the combinations of the a_i that the toy's three monomials determine least well.

Factorial test (`code/density_ablation.py`). To find what caused the old ensemble's spread between bin centres, and whether the density target pays off, the same network was trained with each ingredient changed separately.

Simple toy (the toy above; 9 configurations, 3 seeds each, all trained to convergence the same way). No configuration, the exact old one included (ReLU, linear input, bin values, no ϕ: χ^2 per bin 0.997), shows a larger seed spread between bin centres than at them (ratio 0.93–1.14). In an easy problem, neither ReLU nor the linear input makes the network wiggle.

Realistic toy. The truth is the spline model's own 23 $a_i(m)$ (`runs/SMOOTH_J40`), at 1,500 HEFT points of the real training spectra, with 40,000 Poisson events per spectrum: real threshold curvature and real cancellations, and a known answer. Every configuration was trained with the production recipe (L1, mini-batch Adam, 200,000 steps; then the Huber fine-tune with Poisson variances, full-batch Adam, 10,000 steps); with a short recipe this toy does not converge from a random start. Two seeds each; all reach χ^2 per bin 0.97 on the training spectra. The density $d\sigma/dm$ was compared with the truth at 300 HEFT points on a 0.5 GeV grid:

configuration	r.m.s. error / peak (%) 255–600 GeV	255–345 GeV	bias, 255–285 GeV (% of local density)	max seed spread (% of peak), between / at centres, 255–600 GeV
old: ReLU, linear, bins, no ϕ	1.62	3.00	-0.44 ± 0.23	2.40 / 0.22
SiLU, linear, bins, no ϕ	0.53	1.00	-0.79 ± 0.04	1.11 / 0.07
SiLU, sqrt input, bins, ϕ	0.54	1.04	-1.13 ± 0.01	0.30 / 0.06
SiLU, sqrt input, density, ϕ	**0.28**	**0.50**	$\mathbf{-0.02 \pm 0.01}$	**0.21 / 0.09**

What the factorial test shows

- **The realistic toy reproduces the old wiggles**: for the old configuration the seeds agree to 0.13% (of the local density) at the bin centres and differ by up to 4.8% between them, as in `docs/paper/fig_interpolation.pdf` (0.19% and 4.4%).
- **Their causes, in order**: the linear input together with no threshold factor (the maximum spread between centres falls from 1.1% to 0.30% of the peak), ReLU (from 2.4% to 1.1%), and the bin-value target (from 0.30% to 0.21%). ReLU is a cause, but not the main one.
- **The density target halves the error** of the best bin-value network (0.28% against 0.54% of the peak density below 600 GeV; 0.50% against 1.04% near threshold), and **removes a bias** of -1.1% near threshold, the same in both seeds. The seeds agree far better than either agrees with the truth: the bin-value network's error is systematic, the misreading of bin contents as densities, not noise.
- SiLU is three times more accurate than ReLU with the old input (1.62% against 0.53%).

Caveats: two seeds; the truth is itself a spline times ϕ, which favours the ϕ and square-root-input configurations somewhat; the toy has no model error, unlike POWHEG; ReLU with the square-root input was not run.

Next (on the 5090): `heft_train.ipynb` with `MODEL='density'` (L1, Adam), then `heft_finetune.ipynb` with `VARIANCE_FROM='model'` and the default `OPTIMIZER='adam'` (the recipe tested on the realistic toy; L-BFGS remains an option); compare with the spline model on the test spectra (section 9).

References

[1] P. H. C. Eilers, *Ill-posed problems with counts, the composite link model and penalized likelihood*, Statistical Modelling **7** (2007) 239.

[2] G. H. Golub and J. H. Welsch, *Calculation of Gauss quadrature rules*, Math. Comp. **23** (1969) 221.

[3] V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell and G. Wetzstein, *Implicit neural representations with periodic activation functions*, NeurIPS 2020, arXiv:2006.09661.

[4] P. H. C. Eilers and B. D. Marx, *Flexible smoothing with B-splines and penalties*, Statist. Sci. **11** (1996) 89.